BatchOPS

The FAANG Interview Handbook

Event-driven data pipeline · Java 21 / Spring Boot 3 · Kafka · Postgres · S3

What this is

A study guide for explaining and defending BatchOPS in SWE-1 / SDE-1 interviews. Every section gives you the concept, the exact place it lives in your project, likely interviewer questions, a model answer you can say out loud, the follow-ups they will chase, and the traps to avoid. Part F is a pure CS-fundamentals refresher tied back to the project.

Read this first: one honest caveat

The domain core of the backend (validators, coercion, processing plan, summaries, state machines, auto-fixes) is unit-tested: 45 tests pass. The Spring, Kafka and JPA wiring was written carefully but **could not be compiled in the build sandbox** (Maven Central was blocked). Before any interview, run `mvn verify` and `docker compose up` on your machine, fix whatever surfaces, and ideally run one upload end to end. Being able to say "I wrote it, I ran it, here is what broke and how I fixed it" is far stronger than a perfect-sounding design you never ran.

Contents

Part	Topic
A	Pitch and architecture: the story you tell in the first five minutes
B	Kafka and event-driven design deep dive
C	Database and consistency deep dive
D	Application design: errors, incidents, scheduler, security, API
E	Java, JVM and Spring internals
F	CS fundamentals refresher, mapped to the project
G	System design follow-ups: scaling BatchOPS 100x
H	Honest trade-offs, limitations and what you would do next
I	Behavioral stories (STAR) from this project
J	Rapid-fire bank and one-page cheat sheet

Part A · Pitch and architecture

A1. How to use a project in an interview

Interviewers use a project to test three things: **did you really build it** (depth), **do you understand why** (judgment), and **can you communicate** (clarity). They will pick one sentence you say and dig three levels down. So every claim you make must be something you can defend one level deeper. Do not claim Kafka exactly-once semantics if you implemented idempotent consumers; say precisely what you built. Precision is what impresses.

- **Lead with the problem, not the tech.** "Schools upload messy CSV/Excel/PDF files; one bad file must not corrupt reports." Technology is the answer to a problem, never the opening line.
- **Name the failure modes.** Senior-sounding engineers talk about what goes wrong: duplicates, crashes mid-step, poison messages, replicas double-firing.
- **Quantify trade-offs, not just features.** "At-least-once plus an idempotency table" beats "I used Kafka".
- **Admit limits early.** Saying "this is single-broker in compose; in production I would use replication factor 3 and min.insync.replicas 2" shows you know the gap.

A2. The 30-second pitch

> **Memorise this**
>
> "BatchOPS is a data-operations platform. Schools upload student score files in CSV, Excel or PDF; each upload runs through a pipeline of standardise, validate, transform and publish. I rebuilt the original Django backend as an event-driven Spring Boot system: each stage is a Kafka consumer, state changes are published with a transactional outbox so the database and Kafka never disagree, consumers are idempotent so at-least-once delivery behaves like exactly-once, and failures are classified, retried on tiered topics, and dead-lettered. Failures also become deduplicated incidents with suggested fixes, and moderators approve the destructive ones."

A3. The 2-minute pitch (problem, design, proof)

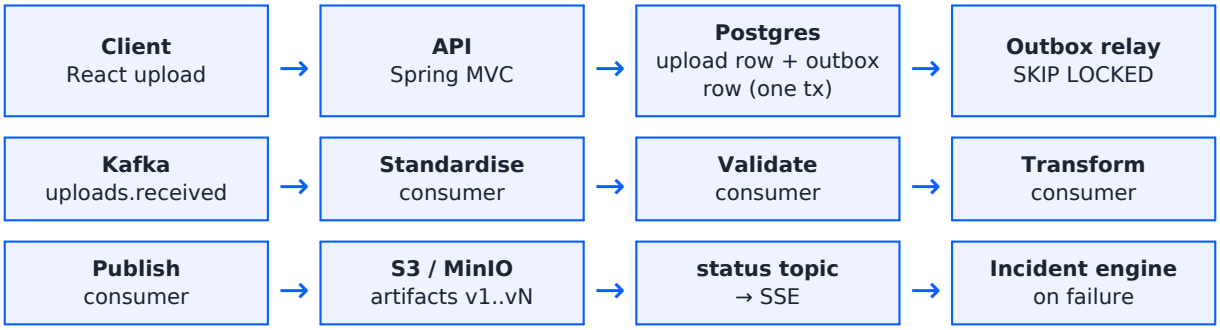
- **Problem.** The original was synchronous Django with a Redis/RQ queue. I audited it and found concrete weaknesses: a dual write between the database and the queue, no idempotency, a blocking sleep-based retry, the original file being overwritten by transformed output, scheduler jobs executed from a dotted path string, and no de-duplication of repeated errors.
- **Design.** Four stage consumers in separate consumer groups, topics keyed by `upload_id` so one upload is always processed in order, an outbox relay that publishes using `FOR UPDATE SKIP LOCKED`, an idempotency table inserted in the same transaction as side effects, and versioned immutable artifacts in S3/MinIO.
- **Operations.** Non-blocking retry tiers (5s, 1m, 10m), a DLQ with replay, incident fingerprinting with a partial unique index, a stuck-run reaper, Prometheus metrics, and an SSE status feed so the UI is live.
- **Proof.** A pure, framework-free domain package with 45 unit tests; the API stays wire-compatible with the existing React frontend (snake_case JSON, DRF-style pagination, `Authorization: Token`).

A4. The problem statement and the original weaknesses

Original weakness (Django)	Why it is a real problem	Upgrade in Spring version
Write DB row, then enqueue job (dual write)	Crash between the two: row says "queued", no job exists. Or job runs before the commit is visible.	Transactional outbox: event row committed with the state change; relay publishes to Kafka
No idempotency	Redelivery or retry reapplies side effects: duplicate incidents, double publishes	`processed_events(consumer,event_id)` inserted in the same transaction; version check skips stale messages
Blocking sleep retry	Holds a worker thread, stalls unrelated work	Retry topics (5s/1m/10m) with a not-before header; poison messages go to DLQ
Original file overwritten	Cannot reproduce, audit or re-run a stage	Immutable source plus versioned artifacts per stage
Every error opens a new incident	Alert storms, unreadable queue	SHA-256 fingerprint of upload+step+normalised error; counter on the live one
Scheduler runs dotted-path strings	Code execution risk; two replicas fire the same job	Task-name allow-list; `job_fires` unique insert dedupes across replicas
Weak auth hygiene	Tokens stored raw, reset codes guessable	SHA-256 hashed expiring tokens, BCrypt, login throttle, hashed reset codes with attempt limit

A5. Architecture walkthrough

Draw this on the whiteboard in this order. Practice until you can do it in under two minutes.



Topic	Purpose	Key
uploads.received → standardized → validated → transformed → published	The happy-path stages; each consumer reads one and emits the next via the outbox	upload_id
uploads.failed	Terminal business failure (e.g. validation rejected), drives incident creation	upload_id
uploads.status	Every state change; the SSE listener fans it out to browsers	upload_id
retry.5s / retry.1m / retry.10m	Delayed re-delivery tiers for transient errors	upload_id
dlq	Poison or retries-exhausted messages, stored for inspection and replay	upload_id

Say this to impress

Why key by upload_id? Kafka guarantees order only within a partition, and the key decides the partition. Keying by upload_id means all events for one upload land on one partition and are processed in order, while different uploads are processed in parallel across partitions. That is the exact trade you want: ordering where it matters, parallelism everywhere else.

A6. Package map (know where everything lives)

Package	Responsibility	Why it is separate
domain	Table, validators, coercion, processing plan, summary builder, state machines, retry policy, fingerprint, cron support, error catalogue	Pure Java, no Spring: fast to test, portable, no hidden I/O
persistence	13 JPA entities and repositories	Isolates Hibernate concerns
events	Outbox, relay, idempotency guard, Kafka config, listeners, DLQ service	All messaging mechanics in one place
pipeline	PipelineService, StageProcessor, UploadService, artifact keys	Orchestration and transactions
incident	IncidentEngine, services, timelines	Failure handling and moderator workflow
schedule	Scheduler, executor, task registry, stuck-run reaper	Time-based work
security, api, io, storage, ops, mail, sse, seed, config	Filters and auth, REST controllers, file readers, object store, metrics, mail, SSE, seed data, config	Standard layering

Say this to impress

If asked "how do you structure a service", say: "I keep a functional core and an imperative shell. The domain package has no Spring imports, so 45 tests run in milliseconds with no containers. The shell handles transactions, Kafka and I/O." This is the hexagonal / clean-architecture idea without the jargon overload.

Part B · Kafka and event-driven design

B1. Kafka fundamentals you must know cold

Concept	One-line definition	Where it shows in BatchOPS
Topic	Named, append-only, partitioned log of records	uploads.received, dlq, retry.5s ...
Partition	Unit of ordering and parallelism; a topic is N partitions	Keyed by upload_id
Offset	Position of a record in a partition; consumer tracks its own	Committed after the listener finishes (ack)
Consumer group	Consumers sharing work: each partition goes to exactly one member of a group	Each stage has its own group, so every stage sees every message of its topic
Key	Hashed to choose partition; same key → same partition	upload_id
Retention	Log is kept by time/size, not deleted on read	Allows replay and audit
Rebalance	Partitions reassigned when members join/leave	Why handlers must be idempotent
Replication factor / ISR	Copies of each partition; in-sync replicas	Compose is 1 broker; prod would be RF 3, min.insync.replicas 2, acks=all

Q1. Why Kafka instead of the Redis/RQ queue you had?

RQ is a job queue: a job is popped and gone, there is no replay, no per-key ordering, and no fan-out to several independent consumers. Kafka is a **durable, replayable log**. That gives me three things I needed: (1) ordering per upload through partition keys, (2) several independent consumers (pipeline stages, SSE status feed, audit) reading the same stream with their own offsets, and (3) replay after a bug fix by re-reading or re-publishing from the DLQ.

Honest counterpoint: for a small single-tenant app, Postgres plus a worker (or even SQS) would be simpler. I chose Kafka because the workload is naturally a multi-stage event stream, and because it let me demonstrate real event-driven patterns.

Likely follow-ups: When would you NOT use Kafka? What about SQS or RabbitMQ? How many partitions would you pick and why?

Trap: Saying Kafka is "faster". Throughput is not the reason here; ordering, durability and replay are.

Q2. What delivery guarantee do you have, and how do you explain "exactly once"?

The transport is **at-least-once**: a record can be delivered more than once (consumer crash before offset commit, rebalance, producer retry). I do not claim Kafka gives exactly-once end to end. I get **effectively-once processing** by making the consumer idempotent: before doing work I insert `(consumer, event_id)` into `processed_events` in the **same database transaction** as the side effects. If the insert conflicts, the event was already processed and I skip. If the transaction rolls back, both the marker and the effects roll back, so a retry starts clean.

Kafka also has transactional/idempotent producers and read-committed consumers (EOS). They guarantee exactly-once only inside Kafka (consume-transform-produce). The moment a side effect touches an external system like Postgres or S3, you are back to needing idempotency.

Likely follow-ups: What if the event has no stable id? What if the side effect is an email? What does idempotent producer (enable.idempotence) actually add?

Trap: "Exactly once is impossible" (too absolute) or "Kafka gives exactly-once" (too naive). The precise answer is: exactly-once *delivery* is impossible in general; exactly-once *effect* is achievable with

idempotency or transactions.

```
BEGIN;
  INSERT INTO processed_events(consumer, event_id) VALUES ('validate', :id);   -- unique violation => duplicate, skip
  -- ... validation work, artifact version row, upload.status update ...
  INSERT INTO outbox(topic, key, payload) VALUES ('uploads.validated', :upload_id, :json);
  INSERT INTO outbox(topic, key, payload) VALUES ('uploads.status',    :upload_id, :json);
COMMIT;      -- everything or nothing
```

Q3. Explain the transactional outbox and the dual-write problem.

Dual-write problem: I need to change my database and also publish a message. These are two systems with no shared transaction. If I commit then publish and crash in between, the database changed but no event exists (lost event). If I publish then commit and the commit fails, an event exists for a change that never happened (phantom event). Retrying does not fix either.

Outbox: write the event into an `outbox` table in the *same* transaction as the state change. The database guarantees both or neither. A separate relay process reads unsent rows and publishes them to Kafka, marking them sent. The relay is at-least-once (it may crash after publishing, before marking), which is fine because consumers are idempotent.

The relay selects with `FOR UPDATE SKIP LOCKED`, so multiple relay instances can run safely: each locks a different batch and never blocks on rows another relay holds.

Likely follow-ups: Polling vs CDC (Debezium)? What about ordering across outbox rows? How do you clean the table? What is the latency cost?

Trap: Forgetting that the outbox relay itself can duplicate. Duplicates are expected; idempotent consumers are the other half of the pattern.

> **Say this to impress**
>
> Upgrade path: "The poll-based relay adds tens of milliseconds and some DB load. In production I would switch to CDC with Debezium reading the Postgres WAL, which removes polling and preserves commit order, without changing the outbox table or the consumers."

Q4. Why does your retry use separate topics instead of a sleep in the consumer?

Sleeping in a consumer thread blocks that partition: every later message for that partition waits, even unrelated uploads that hash to it, and a long sleep risks exceeding `max.poll.interval.ms` which triggers a rebalance. So on a transient failure I publish the message to `retry.5s`, `retry.1m` or `retry.10m` with a not-before timestamp header, and the main consumer moves on. The retry listener holds the record (nack with sleep until the not-before time) and then sends it back to the original stage topic.

Honest detail: the retry-tier listener does wait on its own small consumer, so a tier consumer is delayed, but it only blocks other retries of the same tier, never the main flow. The tiers are exponential backoff: 5s, 1m, 10m. After the tiers, a transient error goes to the DLQ: at most 4 executions in total.

Likely follow-ups: Why exponential backoff? What is jitter and why add it? What about ordering when a message is retried but the next event for the same upload already moved on?

Trap: Claiming retry is fully non-blocking. Say what really happens: it does not block the main pipeline partitions.

Q5. How do you handle out-of-order or stale messages caused by retries?

Each upload has a `version`. Events carry the version they were produced for. If a retried message arrives after the upload already advanced (or was re-uploaded and re-versioned), the handler compares versions and skips the stale message. Combined with the per-key partition order, this means a slow retry can never overwrite newer state.

Likely follow-ups: Is this optimistic locking? (Yes in spirit: version as a fencing token.) What is a fencing token?

Q6. What is a poison message and what does your DLQ do?

A poison message is one that can never succeed: malformed payload, permanently missing file. Retrying forever would block or waste capacity. I classify errors (`TRANSIENT` vs `PERMANENT`). Permanent errors fail the upload immediately and open an incident. Transient errors retry through the tiers and, when exhausted, are parked in the DLQ with the original topic, error and attempt count preserved. Operators can inspect and **replay** from the DLQ after fixing the cause.

Likely follow-ups: How do you alert on DLQ depth? What metadata does a DLQ record need? How do you avoid replay loops?

Q7. How many partitions and consumers? What limits parallelism?

Parallelism per consumer group is capped by the partition count: with P partitions, at most P consumers in a group do useful work. I would start at 6 to 12 partitions per topic, which is more than the current consumer count but cheap, because you can add consumers but repartitioning later changes key-to-partition mapping and breaks per-key order for in-flight data. Throughput math: if a stage takes 200 ms per upload, one consumer handles 5/s; 6 partitions with 6 consumers gives about 30/s.

Likely follow-ups: What if one upload is huge (hot key)? What about consumer lag as a scaling signal (KEDA)?

Trap: Saying "add more consumers" without mentioning the partition ceiling.

Q8. What are the consumer settings that matter and what happens on rebalance?

`enable.auto.commit=false` with manual ack after the DB transaction commits, so a crash re-delivers instead of losing. `auto.offset.reset=earliest` for new groups. `max.poll.records` and `max.poll.interval.ms` bound how long processing can take before the broker assumes the consumer is dead. On rebalance, uncommitted records are re-delivered to the new owner, which is exactly the case the idempotency table protects.

Ack-after-commit means a crash between commit and ack causes one duplicate; ack-before-commit would cause a lost message. I prefer duplicates plus idempotency over loss.

B2. Event design questions

Q9. Why one topic per stage rather than one topic with an event type?

Separate topics give independent consumer groups, scaling and lag per stage, clean ACLs, and simple retention per stage. The downside is more topics to manage. A single-topic design suits low volume and strict global ordering across stage types, but couples consumers. Given four independently scalable stages, topic-per-stage is the better fit.

Q10. What would schema evolution look like?

Today payloads are JSON. For production I would use Avro or Protobuf with a schema registry, backward-compatible changes only (add optional fields with defaults), and consumers that ignore unknown fields. The rule: deploy consumers first, then producers, so nobody reads a format it does not understand.

Q11. How do you observe the pipeline?

Micrometer exposes Prometheus metrics: stage durations, counts per outcome, retry counts, DLQ count, outbox lag, and consumer lag via Kafka metrics. An OpenTelemetry bridge propagates trace ids through headers so one upload is one trace across stages. The SSE feed gives users live status. Golden signals to alert on: outbox age, consumer lag growth, DLQ rate, stage error ratio.

> **Careful / common trap**
>
> Do not over-claim the Kafka features you did not use: Kafka Streams, exactly-once transactions (EOS), schema registry and Debezium are **upgrade ideas**, not implemented. Saying "here is what I implemented, and here is what I would add" is credible; listing buzzwords you cannot explain is not.

Part C · Database and consistency

C1. ACID, isolation and the anomalies

Isolation level	Prevents	Still allows	Postgres note
Read Uncommitted	nothing	dirty reads	Treated as Read Committed
Read Committed (default)	dirty reads	non-repeatable read, phantom, write skew	Each statement sees a fresh snapshot
Repeatable Read	dirty, non-repeatable (and phantoms in PG)	write skew	Snapshot taken at first statement; may fail with serialization error
Serializable	all anomalies	nothing (may abort)	SSI; retry on 40001

- **Atomicity**: stage effects plus marker plus outbox rows commit together. **Consistency**: constraints (unique, FK) protect invariants. **Isolation**: concurrent workers do not see partial state. **Durability**: WAL fsync before commit returns.
- **MVCC**: Postgres keeps row versions; readers do not block writers. That is why polling the outbox with SKIP LOCKED is cheap.

Q12. Why is unique-constraint insert a better idempotency check than "SELECT then INSERT"?

Select-then-insert is a check-then-act race: two workers both see "not present" and both proceed. A unique constraint makes the database serialise the decision: exactly one insert wins, the other gets a violation. Correctness moves from application code (racy) to a database invariant (atomic). The same principle applies to the dedupe index, the live incident fingerprint and `job_fires`.

Likely follow-ups: What if you used Redis SETNX? (Works but the marker and side effects are in different systems: no atomic commit.)

Q13. Explain FOR UPDATE SKIP LOCKED. Why not just FOR UPDATE, or an application lock?

`SELECT ... FOR UPDATE` locks the selected rows; a second transaction selecting the same rows blocks until the first finishes. `SKIP LOCKED` makes the second transaction silently skip rows that are already locked and take the next ones. That turns a table into a safe multi-consumer work queue with no blocking and no double-claiming. An application-level lock needs a lock service and fails awkwardly on crashes; the database row lock is released automatically when the transaction ends or the connection dies.

Likely follow-ups: Does it guarantee fairness or ordering? (No: skipped rows can be taken out of order, so strict ordering needs one relay or a per-key scheme.) What about starvation?

Trap: Forgetting the trade-off: SKIP LOCKED gives up strict ordering across rows. I rely on per-upload ordering from the Kafka key, not from the outbox scan.

```
SELECT * FROM outbox
WHERE sent_at IS NULL
ORDER BY id
LIMIT :batch
FOR UPDATE SKIP LOCKED;     -- native query in OutboxRepository.lockNextBatch
```

C2. Indexes, B-trees and partial unique indexes

An index is a separate sorted structure (usually a B+ tree) that maps column values to row locations. Lookup is O(log n) with a high fan-out (hundreds of keys per page), so a billion rows needs only about 4 to 5 page reads. Costs: extra storage and slower writes (every insert or update maintains the index).

- **Composite index order matters.** An index on (department, sha256) serves queries filtering on department, or department and sha256, but not sha256 alone (leftmost-prefix rule).
- **Partial index** covers only rows matching a predicate. Smaller, faster, and, with UNIQUE, it expresses conditional uniqueness.
- **Covering index / index-only scan**: all needed columns are in the index, so the table is not touched.
- **Selectivity**: an index on a low-cardinality column (status with 5 values) is often ignored by the planner.

Your index	What it enforces	Why partial
`uq_uploads_dedupe` on (department, sha256) WHERE status <> 'failed'	The same file cannot be uploaded twice into a department…	…but a failed upload may be retried with the same content
`uq_incident_live_fingerprint` on (fingerprint) WHERE state <> 'resolved'	At most one *live* incident per fingerprint; repeat errors bump a counter	Once resolved, the same error later is a *new* incident

Q14. How does the partial unique index solve incident de-duplication race-free?

Two workers fail on the same upload step with the same normalised error at the same moment. Both compute the same SHA-256 fingerprint and try to insert an incident. The partial unique index lets exactly one insert win; the loser catches the violation and increments the occurrence counter on the existing live row instead. No application lock, no duplicated incident, and once someone resolves it the predicate no longer matches, so a later recurrence creates a fresh incident.

Likely follow-ups: Why hash and not store the raw string? How do you normalise the error (strip ids, numbers, paths)? What if normalisation is too aggressive?

C3. CAP, PACELC and consistency models

- **CAP**: under a network *partition* you choose consistency or availability; you never get all three. Postgres (single primary) chooses C over A: if the primary is unreachable, writes fail. Kafka lets you tune: with acks=all and min.insync.replicas=2 it favours consistency; with acks=1 it favours availability.
- **PACELC** adds: Else (no partition) you still trade Latency vs Consistency, e.g. synchronous replication is safer but slower.
- **Eventual consistency in BatchOPS**: the UI sees status through Kafka and SSE slightly after the DB commit. That is acceptable because the DB is the source of truth and any client can fetch the real state through REST.
- **Read-your-writes** is preserved for the API because reads go to the primary.

Q15. Where in your system do you accept eventual consistency, and where do you refuse it?

Accept: status propagation to the browser, metrics, downstream stage start (a few hundred ms). Refuse: the decision that an event was processed, the uniqueness of incidents and uploads, job-fire dedupe, and who can do what (authz). Those are protected by transactions and unique indexes. The rule of thumb: *money-like invariants get strong consistency; views get eventual consistency*.

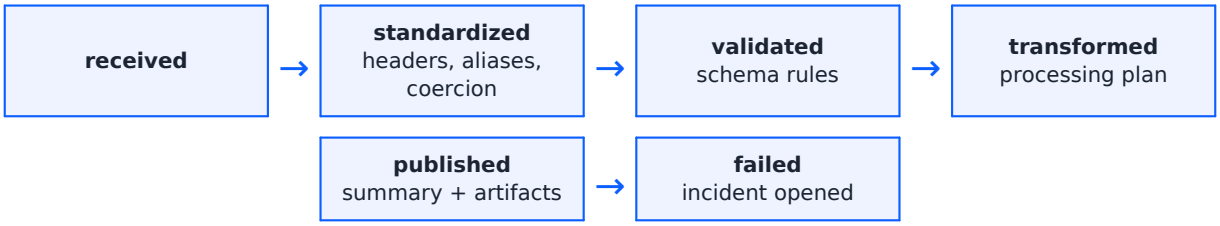
Q16. Hibernate/JPA questions: N+1, lazy loading, jsonb?

N+1: loading a list of parents then one query per child. Fix with fetch joins, `@EntityGraph` or batch fetching. **Lazy proxies**: a lazy association is a proxy object; reading it outside a session throws `LazyInitializationException`, and field access on a proxy can read null. I set `ddl-auto: none` and let Flyway own the schema so production never mutates schema by accident. **jsonb** stores flexible payloads (validation evidence, summaries) with indexing and operators, at the cost of weaker typing.

Likely follow-ups: Open-session-in-view? (Disabled is safer: it prevents accidental queries in controllers.) Flyway vs Liquibase? How do you do zero-downtime migrations (expand/contract)?

Part D · Application design

D1. The pipeline and state machines



Each upload has a status (`UploadStatus`), each incident a state (`IncidentState`) and each ticket a status (`TicketStatus`). Transitions are checked by pure functions in the domain package, so an illegal jump (for example resolved back to open without reopening) is impossible and unit tested.

Q17. Why model states as explicit state machines?

A state machine turns scattered if-statements into a single table of legal transitions. It makes illegal states unrepresentable, makes behaviour testable in isolation, and gives a natural audit trail (every transition is an event). It also protects against concurrent updates: a stale worker trying to move an upload from validated to published when it is already failed is rejected by the transition check and the version number.

Likely follow-ups: How do you persist it? (enum column + version.) How do you make transitions atomic? (single transaction; conditional update).

Q18. Walk me through what happens when a user uploads a file.

1) The API receives the multipart file, hashes it with SHA-256 and checks the dedupe index. 2) It stores the raw bytes in object storage as the immutable source. 3) In **one transaction** it inserts the upload row and an outbox row for `uploads.received`. 4) The relay publishes the event. 5) The standardise consumer reads it, runs header normalisation, alias mapping and numeric coercion, writes artifact v1, and in one transaction marks the event processed, updates the upload and writes outbox rows for the next stage and for status. 6) Validate, transform and publish repeat the same shape. 7) The status topic feeds SSE so the browser updates live. 8) On failure, the failure handler classifies the error and either schedules a retry tier or fails the upload, parks in DLQ if retries are exhausted, and opens or bumps an incident.

Likely follow-ups: Where could it crash, and what happens at each point? (Prepare a failure-point table: Section D7.)

D2. Error classification and retries

Class	Examples	Behaviour
PERMANENT	FileNotFoundException, IllegalArgumentException, validation rejected	No retry; upload fails; incident opened
TRANSIENT	Timeouts, S3 or DB blips, broker hiccups	Retry via 5s → 1m → 10m tiers; then DLQ
Unknown	Anything unclassified	Treated as transient (fail-safe toward retry, bounded by tiers and DLQ)

Q19. How do you decide whether an error is retryable?

Retry only when repeating the *same* action could plausibly succeed. A missing file or bad argument will fail identically forever, so it is permanent. A timeout might not. For unknown errors I default to transient, because wrongly giving up loses work while wrongly retrying is bounded by the tier count and ends in the DLQ. Important subtlety: retries are only safe because the work is idempotent.

Likely follow-ups: Retry budgets and circuit breakers? Retry storms? Why exponential backoff with jitter?

Trap: Retrying permanent errors (wastes resources and delays the incident).

> **Key idea**
>
> Exponential backoff spreads load: if a dependency is down, immediate retries from every worker make recovery harder. Jitter (random spread) prevents synchronised retry waves (the thundering herd). Mention both even though the current tiers are fixed; naming jitter as the next improvement is a strong answer.

D3. Incident engine, known errors and auto-fix

When an upload fails, `IncidentEngine` normalises the error text, computes a SHA-256 fingerprint over upload id, step and normalised error, then opens an incident or bumps the occurrence counter on the live one. It matches the message against a **known-error catalogue** (regex to cause and suggested fix). Fixes come in two kinds.

Fix kind	Examples	Policy
Safe / reversible	Trim whitespace, normalise headers, map aliases, strip thousands separators	Applied automatically; new artifact version; original untouched
Destructive / lossy	Drop duplicates, clip out-of-range scores, drop rows with empty keys	Needs moderator approval via `/approve-fixes`; fully logged in the timeline

Q20. Why require approval for some fixes?

Automation is great until it silently changes data. Dropping duplicate rows or clipping a score of 450 to 100 alters meaning and could hide a real problem (maybe the marks are out of 500). So reversible, information-preserving fixes are automatic; information-destroying fixes are a human decision. Because the source file is immutable and each fix creates a new artifact version, even approved changes are reproducible and auditable.

Likely follow-ups: How would you roll back an approved fix? (Reprocess from source or an earlier version.) How do you prevent a catalogue regex that never matches? (Reachability unit test.)

> **A detail most candidates never mention**
>
> The known-error catalogue has a **reachability unit test**: every pattern must be triggered by some real message, so a typo in a regex cannot silently make a fix dead code. That is a neat, concrete example of testing your own configuration.

Q21. What exactly is in the fingerprint and why?

SHA-256 of upload id + pipeline step + normalised error text (numbers, ids and paths stripped). Including the upload and step means "same problem, same place". Normalising means "row 17 invalid" and "row 203 invalid" collapse together. It is a dedupe key, not a security boundary, so collisions are a non-issue at SHA-256 strength. The fixed-length hash is convenient for an index.

D4. Immutable source and versioned artifacts

The raw upload is never modified. Each stage writes a new object, keyed by upload and version, in S3/MinIO. This is the **event-sourcing-lite** idea: keep the inputs, derive everything else. You get reproducibility (rerun any stage), auditability, safe retries (writing the same key twice is idempotent) and simple rollback. Cost: more storage, mitigated by lifecycle rules.

Q22. Why object storage instead of the database or local disk?

Large binary blobs bloat the database and its backups, and local disk does not work with several replicas. Object storage is cheap, durable (11 nines on S3), horizontally scalable and addressable by key. The DB stores metadata and the key; the bytes live in S3. MinIO gives an S3-compatible API locally, and the same SDK v2 code runs against real S3.

Likely follow-ups: How do you avoid orphaned objects? (Write object first, then DB row; a cleanup job removes unreferenced keys.) Multipart upload for big files? Pre-signed URLs?

D5. Scheduler design

Q23. How does the scheduler avoid double-firing with multiple replicas?

A scheduled fire is identified by (job, scheduled_time). Before running, a replica inserts that pair into `job_fires` with a unique constraint. Only the replica whose insert succeeds runs the job; the others get a violation and skip. It reconciles against the database on each tick, so it survives restarts, and a **catch-up window** runs missed fires after downtime without running unbounded history. Tasks are dispatched through a **task-name allow-list** (`TaskRegistry`) rather than executing arbitrary dotted paths, which closes a remote-code-execution hole. A **stuck-run reaper** marks runs that have been "running" too long as failed so nothing hangs forever.

Likely follow-ups: Why not ShedLock or Quartz clustering? (Both valid; mine is the same idea in ~100 lines. In prod I would consider Quartz/ShedLock.) Clock skew between nodes? (Fires are computed from the DB-stored schedule; the unique key is deterministic.)

Trap: Using in-memory flags or "only one replica runs the scheduler" as a design; that is a single point of failure.

D6. Security

Concern	What you did	Why
Password storage	BCrypt (adaptive, salted)	Slow by design; resists GPU brute force; salt defeats rainbow tables
API tokens	256-bit random token, only SHA-256 hash stored, with expiry	A DB leak does not leak usable credentials. Random high-entropy tokens do not need a slow hash
Login abuse	Throttle on repeated failures	Stops credential stuffing and online brute force
Reset / verify codes	Stored hashed, attempt limit, expiry	Short codes are guessable, so cap attempts and never store plaintext
Account enumeration	Decoy responses: same reply whether the email exists or not	Prevents attackers discovering valid accounts
Authorisation	Roles plus optional per-department scoping	Least privilege; users see only their department
Session model	Stateless, custom token filter in Spring Security	Scales horizontally, no sticky sessions

Concern	What you did	Why
Scheduler execution	Allow-listed task names	No arbitrary code path execution

Q24. Why BCrypt for passwords but plain SHA-256 for tokens?

Passwords are low-entropy and human chosen, so an attacker with the hash can guess billions per second against a fast hash. BCrypt is deliberately slow and salted. API tokens are 256 bits of randomness; there is nothing to guess, so the only risk is leakage of the stored value, which a one-way hash handles, and a fast hash keeps token verification cheap on every request.

Likely follow-ups: What is a salt? What is a pepper? Argon2 vs BCrypt? JWT vs opaque tokens? (Opaque tokens are revocable with a single delete; JWTs are self-contained but hard to revoke.)

Q25. How do you protect against OWASP top issues?

Injection: JPA parameter binding and native queries with bind parameters. Broken auth: hashed expiring tokens, throttling. Broken access control: role and department checks in the service layer, not only in the UI. Security misconfiguration: secrets from environment, Flyway-owned schema. Insecure deserialisation and code execution: task allow-list, no dotted-path imports. XSS/CSRF: JSON API with header-token auth is not cookie-based, so CSRF does not apply; the React frontend escapes output.

Likely follow-ups: CORS? Rate limiting at the gateway? Secrets management (Vault, KMS)?

D7. Failure-mode table (the most impressive page in this book)

Interviewers love "what if it crashes here". Memorise this table.

Crash / failure point	Result	Why it is safe
API crashes before commit	Nothing saved; client gets an error and retries the upload	One transaction: row and outbox together or not at all
API commits, relay not yet run	Event waits in the outbox	Relay picks it up later; nothing lost
Relay publishes, crashes before marking sent	Event published twice	Consumers are idempotent (processed_events)
Consumer crashes mid-work	Transaction rolls back; offset not committed	Redelivery starts from clean state
Consumer commits DB, crashes before ack	Message redelivered	processed_events marker already committed, so it is skipped
Two consumers get the same message (rebalance)	One inserts the marker, the other hits the unique violation	Database serialises the decision
Transient dependency outage	Retry tiers 5s, 1m, 10m	Non-blocking; bounded
Retries exhausted	Parked in DLQ with context; incident opened	Replayable after a fix
Stale retry arrives after newer version	Skipped by version check	Version acts as fencing token
Two replicas run the same cron tick	One wins job_fires insert	Unique constraint
Run hangs forever	Stuck-run reaper marks failed	Nothing stays "running" indefinitely
Kafka down	API still accepts uploads; outbox accumulates	Decoupled by the outbox; relay drains on recovery (watch outbox age)

D8. API design

The frontend was already written against a Django REST Framework API, so the Spring backend reproduces the wire contract: snake_case JSON, `{count,next,previous,results}` pagination with `page_size` up to 5000, `?ordering=` (with a leading minus for descending), `Authorization: Token <t>`, and trailing-slash tolerance so `/api/uploads` and `/api/uploads/` both work.

Q26. REST design: what makes an API good? What about pagination and idempotency?

Resources as nouns, correct verbs and status codes (201 create, 204 delete, 400 validation, 401 unauthenticated, 403 forbidden, 404, 409 conflict, 429 throttle), stable contracts, pagination for lists, filtering and ordering through query params. **Safe** methods (GET) have no side effects; **idempotent** methods (PUT, DELETE) give the same result when repeated; POST is not idempotent, which is why uploads are deduplicated by content hash (an idempotency key of sorts). Offset pagination is simple but slow for deep pages and unstable under inserts; keyset (cursor) pagination is better at scale. I kept offset because the existing frontend depends on it.

Likely follow-ups: How do you version an API? (URL or header; additive changes are non-breaking.) PATCH vs PUT? Rate limiting algorithms (token bucket, sliding window)?

Q27. SSE vs WebSocket vs polling for live status?

Status updates are one-directional (server to browser), so **Server-Sent Events** fit: plain HTTP, automatic reconnect, event ids for resume, works through most proxies, and simpler than WebSockets. WebSockets are bidirectional and right for chat or collaborative editing. Long polling is a fallback. Caveats: browsers limit SSE connections per domain on HTTP/1.1 (about 6), so HTTP/2 helps; each connection holds a server resource, so the emitter needs timeouts, cleanup on disconnect and, with several replicas, a fan-out source like the Kafka status topic (which is why status is a topic).

Likely follow-ups: How do you scale SSE to 100k clients? (Dedicated gateway, each node consumes status topic and pushes to its local connections; sticky routing not needed.)

Part E · Java, JVM and Spring internals

E1. Spring questions every interviewer asks

Q28. What is dependency injection and inversion of control? Why use them?

Instead of a class creating its collaborators with new, the container creates them and supplies them (constructor injection). Control of object creation is inverted to the framework. Benefits: loose coupling, easy testing (pass a fake), configuration in one place, lifecycle management. I use constructor injection everywhere: fields are final, dependencies are explicit, and the object cannot exist half-built.

Likely follow-ups: Field vs constructor vs setter injection? Singleton vs prototype scope? Circular dependencies?

Q29. How does @Transactional work, and what is the self-invocation problem?

Spring wraps the bean in a **proxy** (JDK dynamic proxy or CGLIB subclass). A call from outside goes through the proxy, which opens a transaction, calls the real method, then commits or rolls back. If a method inside the same class calls another `@Transactional` method via `this`, it bypasses the proxy and the annotation does nothing. This matters in my code: `handleFailure` must run in a **fresh transaction** after the main one rolled back (a rolled-back transaction cannot record the failure), so it is invoked across a bean boundary, using `REQUIRES_NEW`, so the proxy applies.

Other gotchas: by default only unchecked exceptions trigger rollback (use `rollbackFor` for checked ones); private or final methods cannot be proxied by CGLIB; transactions are bound to a thread, so work handed to another thread is outside them.

Likely follow-ups: Propagation types (REQUIRED, REQUIRES_NEW, NESTED)? What does readOnly=true do? What if a transactional method catches its own exception?

Trap: Saying "@Transactional is magic". Explain proxies; that is the whole answer.

Q30. What happens at application startup in Spring Boot?

`SpringApplication.run` creates the application context, performs component scanning, applies auto-configuration (conditional on classpath and properties), instantiates singleton beans, runs Flyway migrations, starts the embedded Tomcat and the Kafka listener containers. Profiles and environment variables override properties, which is how the same jar runs with different roles.

Likely follow-ups: How do role-split processes work? (`BATCHOPS_ROLES_*` toggles which beans load: api, worker, scheduler, so one artifact scales each role separately.)

Q31. Explain the role-split deployment.

One codebase, one image, three roles controlled by properties: the **api** role serves HTTP, the **worker** role runs Kafka consumers and the outbox relay, the **scheduler** role runs cron reconciliation. You can scale workers to match partition count without scaling the API, and a slow batch job can never starve HTTP request threads. It is the modular monolith approach: microservice-style operations without microservice-style distributed complexity.

Likely follow-ups: Why not microservices? (Team size and operational cost. Extract only when a boundary has an independent scaling or release need.)

E2. Java fundamentals that show up

Topic	What to know	Project link
equals / hashCode	Equal objects must have equal hashes; HashMap relies on it; records generate both	Records for events and fingerprints
HashMap internals	Array of buckets; collisions become linked list, treeified at 8 entries per bucket; resize at load factor 0.75; average O(1) get/put	Duplicate-key detection in SchemaValidator uses a HashSet
String immutability	Safe to share and cache; enables string pool and hash caching	Normalised column names
Checked vs unchecked exceptions	Checked: caller must handle; unchecked: programming errors	StageException carries TRANSIENT/PERMANENT classification
Streams / Optional / records / sealed types (Java 17-21)	Declarative pipelines, concise immutable data carriers	Domain records
Generics and type erasure	Type parameters are compile-time only	Serialiser configuration
Garbage collection	Generational; G1 default; young collections cheap, full GC pauses are the concern	Large CSVs: stream instead of loading everything

Q32. Explain Java memory: heap, stack, and garbage collection.

Each thread has a **stack** of frames holding local variables and references; frames vanish on return. Objects live on the shared **heap**, managed by the garbage collector, which reclaims objects no longer reachable from GC roots (thread stacks, statics). Generational GC exploits that most objects die young: new objects go to the young generation, survivors are promoted. G1 (default) works in regions and targets pause-time goals. Memory leaks in Java are *reachable but unused* objects: for example static collections or unbounded caches; in my system the risk is holding an entire uploaded file in memory, so I process rows as streams or bounded tables.

Likely follow-ups: What is a memory leak in Java? OutOfMemoryError causes? What is escape analysis? What are virtual threads (Java 21)?

Q33. Threads, concurrency and race conditions: where do they appear in your project?

Kafka listener containers run each consumer on its own thread; the HTTP server uses a thread pool; the relay and scheduler run concurrently. Shared mutable state is the danger. A real bug I found and fixed: `AuthService` held a `lastCode` instance field in a singleton bean, so two concurrent requests could overwrite each other. The fix was to return an immutable `Issued` record instead of stashing state in the singleton. General rules: singletons must be stateless; prefer immutability; use the database (constraints, locks) for cross-process coordination because in-JVM locks do not span replicas.

Likely follow-ups: synchronized vs ReentrantLock vs atomic classes? volatile? What is a deadlock and how do you prevent it (consistent lock ordering)? ConcurrentHashMap internals?

> **Your best concurrency story**
>
> Telling the `lastCode` story is gold: it is a real race you found in your own code, with root cause and fix. It proves you read and review your code critically. Use it in the "tell me about a bug" question (see Part I).

Q34. What is a virtual thread and would you use it here?

Java 21 virtual threads are cheap, JVM-scheduled threads multiplexed onto a few OS threads; blocking I/O parks the virtual thread rather than a platform thread. They suit high-concurrency I/O-bound request handling (many API calls waiting on S3 or DB). For Kafka consumers, concurrency is bounded by partitions, so they help less. Caveats: pinning inside synchronized blocks, and CPU-bound work gains nothing. I would try them on the API role once measured.

E3. Testing strategy

Layer	What	Tool	Status
Unit (domain)	Validators, coercion, plan, summary, fixes, state machines, fingerprint, cron	JUnit 5	45 tests pass
Storage	LocalObjectStore behaviour	JUnit 5	Passing
Integration (next)	Repositories, Flyway migrations, outbox SKIP LOCKED, idempotency under concurrency	Testcontainers (Postgres, Kafka)	To run / add locally
Contract (next)	API wire compatibility with frontend	Spring MockMvc / JSON snapshots	To add
End to end (next)	Upload file through every stage, fail and recover	Docker compose	Run manually first

Q35. How did you test, and what would you test next?

Test pyramid: many fast unit tests over the pure domain core (45), fewer integration tests, very few end-to-end. I designed the core to have no framework dependencies precisely so it is easy to test. What I have not yet done is integration-test the concurrency guarantees: I would add Testcontainers tests that (1) fire the same event twice concurrently and assert one effect, (2) run two relays and assert no row is published twice from the same lock, and (3) kill a consumer mid-transaction and assert clean redelivery. Those tests are the evidence for the claims in this handbook.

Likely follow-ups: Mocks vs fakes? Flaky tests? Property-based testing for the validator? Chaos testing?

Trap: Claiming you tested the Kafka and JPA layers when you did not. Say clearly what is tested and what is next.

E4. Docker and deployment

`docker-compose.yml` starts Postgres, Kafka, MinIO, Prometheus, the backend and the frontend. The backend Dockerfile is multi-stage: Maven builds the jar in a builder image; the runtime image contains only a JRE and the jar, so it is smaller and has a reduced attack surface. Configuration comes from environment variables (12-factor), secrets are never baked into the image.

Q36. How would you deploy this to production?

Containers on Kubernetes (or ECS). Separate deployments per role (api, worker, scheduler) with independent replica counts; readiness and liveness probes through the actuator endpoints; managed Postgres (RDS) with read replicas and backups; managed Kafka (MSK/Confluent) with RF 3; S3 instead of MinIO; secrets from a secrets manager; Prometheus and Grafana for metrics and alerts; blue-green or rolling deploys with backward-compatible migrations (expand, then contract). Workers autoscale on consumer lag, API on CPU or request rate.

Part F · CS fundamentals, mapped to the project

SDE-1 interviews test fundamentals first and projects second. The trick is to answer the fundamental cleanly and then **anchor it in BatchOPS** in one sentence. That shows you know the theory and have used it.

F1. Data structures and algorithms inside BatchOPS

Where	Structure / algorithm	Complexity	Say this
Duplicate student_id detection	HashSet of seen keys	O(n) time, O(n) space	One pass beats comparing every pair ($O(n^2)$)
Duplicate detection per subject	HashSet on composite key (id, subject)	O(n)	Composite key: same student in different subjects is legal
Header alias mapping	HashMap normalised name → canonical name	O(1) lookup per column	Precedence order resolves "grade" → score before class
Numeric column detection	Single scan counting numeric cells, 60% threshold	O(rows × cols)	Heuristic; identifier columns are excluded by name
Summary statistics	Mean, min, max in one pass; median/percentile need sort	O(n) and O(n log n)	Quickselect gives median in O(n) average if needed
Outbox scan	B-tree index on (sent_at, id)	O(log n + batch)	Index makes "next unsent" cheap
Fingerprint	SHA-256 hash	O(length)	Fixed-size key for index and dedupe
Retry tiers	Fixed array [5s, 1m, 10m]	O(1)	Exponential growth; add jitter next
Cron next-fire	Field-by-field matching over time	Bounded search	Uses a validated cron parser in CronSupport

Q37. Find duplicates in an array. Compare approaches.

Brute force pairwise: $O(n^2)$ time, O(1) space. Sort then scan adjacent: O(n log n) time, O(1) extra space (or O(n) if you copy). HashSet: O(n) average time, O(n) space. Choose by constraints: when memory is tight and modifying order is fine, sort; otherwise hash. For data that does not fit in memory, use external sort or partition by hash and dedupe each partition, which is also what Kafka key-partitioning gives you for free. My validator uses the HashSet approach, and treats blank keys separately so they are reported as "empty" rather than as duplicates.

Likely follow-ups: What about approximate dedupe for huge streams? (Bloom filter: no false negatives, tunable false positives.)

Q38. How would you compute the median or a percentile of a column of a million numbers?

Sort and index: O(n log n). Quickselect: O(n) average, $O(n^2)$ worst case, O(n) expected with random pivot. For streaming or distributed data: two heaps for a running median, or sketches (t-digest, HdrHistogram) for approximate percentiles. Percentile definition matters (nearest-rank vs linear interpolation); state which one you use.

Q39. Implement an LRU cache (classic follow-up).

HashMap from key to node plus a doubly linked list ordered by recency: get moves the node to the head, put inserts at the head and evicts the tail when full. All operations O(1). In Java, `LinkedHashMap` with accessOrder=true and overridden `removeEldestEntry` does it in a few lines. Project link: a cache for repeated department or user lookups.

F2. Databases

- **Normalisation**: remove redundancy (1NF atomic columns, 2NF no partial dependency, 3NF no transitive dependency). Denormalise deliberately for read speed. jsonb columns are a controlled denormalisation for flexible evidence payloads.
- **Joins**: nested loop, hash join, merge join; the planner picks by size and indexes. Use EXPLAIN ANALYZE to see reality.
- **Transactions and locks**: row locks, deadlocks (two transactions lock rows in opposite order; fix with consistent ordering), optimistic (version column) vs pessimistic (SELECT FOR UPDATE) locking.
- **WAL**: changes are written to a sequential log first, so a crash replays committed work; also the basis of replication and CDC.
- **SQL vs NoSQL**: relational for strong invariants, joins and ad-hoc queries; NoSQL for scale-out key access patterns, flexible schema. BatchOPS needs invariants (unique dedupe, state transitions), so Postgres is right.

Q40. Optimistic vs pessimistic locking: which do you use where?

Pessimistic (`FOR UPDATE [SKIP LOCKED]`) when contention is high and I must prevent conflict: the outbox queue. Optimistic (version compare) when conflicts are rare and retry is cheap: stale-message detection on uploads. Unique constraints are a third tool: they encode "only one of these may exist" so the database arbitrates races for me.

Q41. What is a deadlock and how would it occur here?

Two transactions each hold a lock the other needs. Example: transaction A updates upload 1 then incident 7; B updates incident 7 then upload 1. Prevent by always touching tables in the same order, keeping transactions short (my stage transaction does no network calls to third parties), and using lock timeouts; Postgres detects deadlocks and aborts one, so callers must be ready to retry, which idempotent handlers allow.

F3. Operating systems

Concept	Key points	Project link
Process vs thread	Process: own address space. Thread: shares heap, own stack. Context switch cost higher for processes	API, worker, scheduler roles as separate processes
Synchronisation	Mutex, semaphore, monitor; race condition, deadlock, starvation	lastCode race; SKIP LOCKED avoids blocking
Memory	Virtual memory, paging, page cache, swapping	Postgres relies on the OS page cache
I/O models	Blocking, non-blocking, async; epoll	Tomcat NIO; virtual threads
File system and fsync	Write to buffer cache; fsync forces to disk	WAL durability; Kafka log segments
Scheduling	Preemptive time slicing, priorities	Thread pools sized to cores for CPU work, larger for I/O

Q42. How many threads should a pool have?

CPU-bound work: about the number of cores (more just adds context switching). I/O-bound work: cores × (1 + wait time / compute time), so a task that waits 9x longer than it computes suits about 10× cores. Always bound the queue, otherwise memory grows without limit; a bounded queue plus a rejection policy is backpressure.

F4. Networking and the web

- **HTTP**: stateless request/response; methods, status codes, headers, keep-alive; HTTP/2 multiplexes streams on one TCP connection. **HTTPS**: TLS handshake (certificate check, key exchange, symmetric session key).
- **TCP vs UDP**: TCP is reliable, ordered, congestion-controlled (three-way handshake); UDP is connectionless and low latency. Kafka, Postgres and HTTP all use TCP.
- **DNS** resolves names to IPs with caching and TTLs; **load balancers** (L4 vs L7) spread requests; **reverse proxy** terminates TLS.
- **REST vs gRPC vs GraphQL**: REST is simple and cacheable; gRPC is binary and fast between services; GraphQL lets clients shape responses.
- **CORS**: the browser blocks cross-origin reads unless the server sends allow headers; it is a browser protection, not server security.

Q43. What happens when you type the frontend URL and upload a file?

DNS lookup, TCP handshake, TLS handshake, HTTP request for the React bundle from the static server, then JS runs and calls the API. The upload is a multipart POST with the token header; the server streams the body, hashes it, stores the object, commits the DB row and outbox, and returns 201 immediately, well before processing finishes. The browser then subscribes to an SSE stream to watch progress. Mention that the heavy work is asynchronous: the API responds quickly and the pipeline continues, which is the point of the design.

F5. Distributed-systems vocabulary

Term	Meaning	Where you use it
Idempotency	Repeating an operation has the same effect as doing it once	Consumers, relay, artifact writes, job_fires
At-most-once / at-least-once / exactly-once	Loss possible / duplicates possible / neither (effect)	At-least-once + idempotency
Backpressure	Slow consumers signal producers to slow down instead of dropping or crashing	Bounded queues, Kafka pull model, batch sizes
Dead letter queue	Quarantine for messages that cannot be processed	dlq topic
Saga / compensation	Sequence of local transactions with undo steps instead of a distributed transaction	The pipeline is a choreographed saga; failure opens an incident instead of rolling back
CQRS / event sourcing	Separate write and read models / store events as truth	Immutable source plus versioned artifacts is the light version
Two-phase commit	Blocking atomic commit across systems; fragile	Avoided; outbox instead
Leader election / consensus	Agree on one coordinator (Raft, Paxos)	Kafka KRaft, Postgres primary; not hand-rolled

Term	Meaning	Where you use it
Consistent hashing	Hash ring that minimises key movement when nodes change	How key→partition could be extended; Kafka uses murmur2 mod N
Circuit breaker	Stop calling a failing dependency for a while	Next improvement around S3 calls

Q44. Why not a distributed transaction (two-phase commit) between Postgres and Kafka?

2PC needs every participant to support a prepare phase, blocks resources while a coordinator decides, and a coordinator failure can leave participants stuck. Kafka does not participate in XA transactions. The outbox gives the same practical outcome (no lost or phantom events) using only a local ACID transaction, at the price of eventual delivery and duplicates that idempotency absorbs.

Q45. What is a saga and is your pipeline one?

A saga is a long-running business process broken into local transactions, with compensating actions for undo. My pipeline is a **choreographed** saga: each stage reacts to the previous stage event. Instead of compensating by undoing, failure moves the upload to failed and raises an incident; thanks to immutable source and versioned artifacts, the "compensation" is simply reprocessing. An orchestrator (a central state machine that commands stages) would make the flow easier to see but adds a central component.

F6. Hashing, encryption and encoding (people mix these up)

Tool	Reversible?	Use	Example here
Hash (SHA-256)	No	Integrity, fingerprints, token storage	File dedupe, incident fingerprint, token at rest
Password hash (BCrypt)	No, slow by design	Passwords	User passwords
Encryption (AES, RSA)	Yes with key	Confidentiality	S3 server-side encryption, TLS in transit
Encoding (Base64, URL)	Yes, no key	Representation only	Transporting binary in JSON

Q46. What are the properties of a good hash function, and what is a collision?

Deterministic, fast, uniform distribution, and for cryptographic hashes also pre-image resistance (cannot find input from output), second pre-image resistance and collision resistance (cannot find two inputs with the same output). A collision is two different inputs with the same hash; by the pigeonhole principle they exist, but for SHA-256 finding one is computationally infeasible (birthday bound about 2^{128} work), so using the hash as a content identity for dedupe is sound.

F7. Design principles

- **SOLID**: single responsibility (StageProcessor processes, PipelineService orchestrates), open/closed (add a stage or a known-error pattern without editing the engine), Liskov, interface segregation, dependency inversion (domain does not depend on Spring).
- **Patterns you can point to**: Strategy (processing modes, retry policy), State (upload/incident/ticket), Template (stage processor), Factory (task registry), Observer (Kafka listeners, SSE), Repository, Outbox.
- **Failure-first design**: assume every call can fail, be duplicated, or arrive late; design the recovery path before the happy path.

Part G · System design follow-ups: scaling BatchOPS 100x

The standard follow-up after any project: "Now imagine 100x the load. What breaks first?" Use the framework below: **clarify, estimate, find the bottleneck, fix it, state the trade-off**.

G1. A back-of-envelope estimate you can do live

Assumption	Number
Uploads per day (scaled target)	1,000,000
Average per second	1,000,000 / 86,400 ≈ 12 uploads/s
Peak factor (exam-results season)	10× → about 120 uploads/s
Average file size	2 MB → peak inbound about 240 MB/s (2 Gbit/s)
Storage per day (source + 4 artifacts)	1M × 2 MB × 5 ≈ 10 TB/day → lifecycle rules, compression and tiering are mandatory
Stage time at 300 ms	One consumer ≈ 3.3/s → about 36 partitions and consumers to absorb 120/s
Events per upload (5 stages + status)	≈ 12 events → about 1,400 events/s at peak, easily within one Kafka cluster

> **Note**
>
> Numbers are illustrative. Interviewers care that you *reason* from assumptions to a bottleneck and state assumptions aloud, not that you hit an exact figure.

G2. What breaks first, and the fix

Bottleneck	Symptom	Fix	Trade-off
Single Postgres primary (outbox + state writes)	Write latency, outbox lag	Batch outbox writes, partition tables by time, then CDC instead of polling; shard by department if needed	Sharding complicates cross-department queries
Outbox table growth	Slow scans, bloat	Delete or archive sent rows; partition by day and drop old partitions	Lose in-DB audit unless archived
Partition count ceiling	Consumers idle, lag	Plan partitions ahead (e.g. 48); split hot departments	Repartitioning breaks key ordering for in-flight data
Hot key (one huge upload)	One consumer busy, others idle	Split large files into chunk events keyed by upload_id + chunk and aggregate	Aggregation step and more state
Large files in memory	OOM, GC pauses	Streaming readers, chunked processing, row limits	More complex code
SSE connections	File descriptors, memory	Dedicated push gateway or managed pub/sub; each node subscribes to status topic	Extra component
Object storage cost	Bill growth	Lifecycle to cheaper tiers, compress, keep only final versions after N days	Slower rollback for old data
Metrics cardinality	Prometheus memory	No per-upload labels; use exemplars and tracing	Less granular metrics

Bottleneck	Symptom	Fix	Trade-off
Noisy tenants	One school starves others	Per-department quotas and rate limits; separate consumer groups or priority topics	Fairness logic

Q47. Design the same system if each school must be fully isolated (multi-tenancy).

Options from cheapest to strongest isolation: a `tenant_id` column with row-level security (shared everything); schema per tenant; database per tenant; cluster per large tenant. Kafka: tenant in the key or headers, optionally topics per tier (large vs small tenants). Add per-tenant quotas, per-tenant encryption keys in S3 (KMS), and tenant-scoped metrics. Choose shared-schema plus RLS for many small tenants and dedicated resources for the few large ones. BatchOPS already has optional per-department scoping; that is the seed of this model.

Q48. How would you make this multi-region / highly available?

Within a region: Kafka RF 3 across availability zones with min.insync.replicas 2, Postgres with synchronous standby and automatic failover, stateless API and workers with several replicas. Across regions: active-passive with asynchronous replication (RPO above zero) is the pragmatic choice; active-active needs conflict resolution and is rarely worth it for a pipeline like this. Define RPO and RTO first; they decide everything else. Idempotent consumers make replaying after a failover safe.

Q49. How would you add real-time analytics (scores dashboards) without hurting the pipeline?

Add another consumer group on `uploads.published` that writes to a read-optimised store (ClickHouse or a Postgres read replica with materialised views). That is CQRS: the write path is unchanged, the read model is rebuilt from the event stream, and it can be replayed from Kafka retention. Eventual consistency between write and read is acceptable for dashboards.

Q50. How would you add a new pipeline stage, such as a virus scan?

Add a topic and consumer group between received and standardized, reuse the stage template (idempotency guard, outbox, error classification), register its error patterns in the catalogue, add metrics. Because stages only know their input and output topics, nothing else changes. That open/closed property is the reason for the topic-per-stage design.

Part H · Honest trade-offs, limits and next steps

A strong candidate volunteers weaknesses before being asked. Below are the ones to own, each paired with the upgrade you would make. Saying these out loud signals engineering maturity.

Limitation	Why it exists	What you would do next
Spring, Kafka and JPA wiring compiled and tested only partly	Build sandbox could not reach Maven Central; 45 domain tests pass; wiring needs a local build	Run mvn verify and compose; add Testcontainers integration tests for idempotency, relay and retries
Outbox relay polls	Simplest correct approach	Debezium CDC reading the WAL
Retry tiers are fixed without jitter	Simplicity	Exponential backoff with jitter and retry budget; per-error policies
JSON payloads, no schema registry	Faster to build	Avro/Protobuf with registry and compatibility checks
Single-broker compose	Local development	RF 3, min.insync.replicas 2, acks=all, rack awareness
Offset pagination	Frontend contract depends on it	Keyset pagination for deep lists
No circuit breaker around S3/DB	Not needed at this scale	Resilience4j circuit breaker, bulkheads, timeouts everywhere
Kafka ordering only per key	By design	Keep per-upload ordering; never rely on global order
Incident catalogue is regex-based	Transparent and testable	Add ML or similarity clustering later; keep regex as the safe baseline
Leftover artifacts on partial failure	Object write happens before DB commit	Orphan sweeper job; writes are idempotent by key
Single database for outbox and business data	Needed for atomic outbox	Acceptable; scale with partitioning and read replicas

Q51. What is the biggest weakness of your design?

Operational complexity relative to the problem size. A queue, an outbox, idempotency tables, retry tiers and a DLQ are a lot of moving parts for what could be a monolith with a job table. I accepted that deliberately to demonstrate correctness under failure, and I would say so in a real team: start with the simplest thing that preserves the invariants (Postgres-backed queue with SKIP LOCKED is often enough) and adopt Kafka when fan-out, replay or throughput demand it. The second weakness is that the integration-level guarantees are not yet proven by automated tests; that is the first thing I would fix.

Likely follow-ups: If you had one more week? (Integration tests, then CDC, then dashboards for lag.)

Q52. If you rebuilt it, what would you do differently?

Write the Testcontainers integration tests first, alongside the code, to prove idempotency and redelivery. Adopt a schema registry from day one. Add a per-tenant quota model earlier. And decide topic and partition counts from a capacity estimate rather than defaults.

Part I · Behavioral stories (STAR) from this project

Use **Situation, Task, Action, Result**. Keep each story to about 90 seconds. All of these really happened while building BatchOPS, which makes them credible. Adjust the wording to your own voice.

Q53. Tell me about a hard bug you found and fixed

Situation. While reviewing the auth service I noticed a `lastCode` instance field in a singleton Spring bean used to hand back the generated reset code.

Task. Make the service safe under concurrent requests.

Action. I reasoned that two simultaneous requests would overwrite each other, so one user could receive another user's code. I replaced the shared field with an immutable `Issued` record returned from the method, so no state lives in the singleton.

Result. The race disappeared by construction. Lesson: singletons must be stateless, and review your own code for shared mutable state.

Q54. Tell me about a time a test or tool caught your mistake

Situation. Error classification treated `FileNotFoundException` as transient, so a permanently missing file would retry through all tiers before failing. A compile check also flagged a multi-catch of two exception types related by inheritance.

Task. Get retry behaviour right: permanent errors must fail fast.

Action. I restructured the handler to rethrow runtime exceptions and wrap checked ones with `RetryPolicy.classify`, then made FileNotFound and IllegalArgument permanent.

Result. Permanent errors now fail immediately and open an incident; transient ones use the tiers. Lesson: classification is a core design decision, not a detail.

Q55. Tell me about a time you worked around a blocker

Situation. The build environment could not reach Maven Central, so I could not compile the Spring modules.

Task. Still deliver something verifiable and be transparent about the rest.

Action. I kept the domain core free of framework dependencies, then compiled and ran it with plain JDK and a stub JUnit, fixing failures such as a wrong test expectation in the summary builder. I documented the exact remaining verification steps.

Result. 45 domain tests pass; the README and project notes say clearly that mvn verify and docker compose still need a local run. Lesson: architecture that isolates the core lets you verify progress under constraints, and honesty about gaps builds trust.

Q56. Tell me about a design decision you made and defended

Situation. The original system used a queue plus direct DB writes, with duplicated side effects possible on retry.

Task. Choose a messaging design that stays correct under crashes.

Action. I compared a simple Postgres job table, RQ, and Kafka. I chose Kafka with a transactional outbox and idempotent consumers because I needed per-key ordering, replay and fan-out, and I documented the simpler alternative and when I would switch back to it.

Result. A design where every crash point has a defined outcome (the failure table in Part D7). Lesson: state the trade-off and the alternative, not just the choice.

Q57. Tell me about a time you had to learn something quickly

Situation. I needed Kafka delivery semantics, the outbox pattern and SKIP LOCKED quickly for this rebuild.

Task. Build a correct implementation, not a demo.

Action. I started from failure modes (what happens if it crashes here?), learned each mechanism as the answer to one failure, and encoded each guarantee as a unique constraint or transaction boundary.

Result. I can explain each mechanism from the problem it solves, which is what this handbook is built on.

Q58. Tell me about a time you had to keep compatibility with something you could not change

Situation. The React frontend was already written against a Django REST Framework API.

Task. Replace the backend without touching the frontend.

Action. I mapped the exact contract: snake_case, DRF pagination envelope, ordering parameters, token header, trailing slashes, and reproduced it in Spring, adding JSON conventions in config rather than in every DTO.

Result. A drop-in replacement. Lesson: treat the wire contract as the real interface and write it down before coding.

Questions you should ask the interviewer

- "How do your teams handle duplicate or out-of-order events in production, and have you standardised on outbox or CDC?"
- "What does on-call look like for your data pipelines, and what are the most common failure modes?"
- "How do you decide when to split a service versus keep a modular monolith?"

Part J · Rapid-fire bank and cheat sheet

J1. Thirty questions with ten-second answers

#	Question	Ten-second answer
1	What is the outbox pattern?	Write the event to a table in the same transaction as the state change; a relay publishes it later. Solves the dual write.
2	Why SKIP LOCKED?	Lets multiple workers claim different rows without blocking each other; turns a table into a work queue.
3	Why key by upload_id?	Same key goes to the same partition, so one upload is processed in order while others run in parallel.
4	Is your system exactly-once?	At-least-once delivery plus idempotent consumers gives exactly-once effect. Not exactly-once delivery.
5	What is idempotency?	Doing an operation twice has the same result as once.
6	How do you make a consumer idempotent?	Insert the event id into a unique table in the same transaction as the side effects.
7	What if the relay crashes after publishing?	The event is published again; consumers skip the duplicate.
8	Retry topics vs sleeping?	Sleeping blocks the partition and risks rebalance; retry topics let the main flow continue.
9	What goes to the DLQ?	Permanent failures and transient ones that exhausted retries, with context, for inspection and replay.
10	Transient vs permanent error?	Transient may succeed on retry; permanent never will.
11	Why a partial unique index?	Enforces uniqueness only for rows matching a predicate: one live incident per fingerprint; one non-failed upload per hash.
12	What does the fingerprint contain?	SHA-256 of upload, step and normalised error text.
13	Why immutable source?	Reproducibility, audit and safe reprocessing; fixes create new versions.
14	Safe vs destructive fixes?	Safe are reversible and automatic; destructive lose information and need moderator approval.
15	How do replicas not double-fire cron?	Unique insert into job_fires per job and scheduled time.
16	Why an allow-list of tasks?	Executing arbitrary dotted paths is a code-execution risk.
17	Why BCrypt?	Slow, salted hash that resists brute force.
18	Why hash tokens at rest?	A database leak does not reveal usable tokens.
19	Why decoy responses?	Stop attackers learning which emails have accounts.
20	SSE vs WebSocket?	SSE is one-way, simple, auto-reconnecting; WebSocket is two-way.
21	What is @Transactional under the hood?	A proxy that opens and commits the transaction; self-invocation bypasses it.
22	REQUIRES_NEW?	Suspends the current transaction and starts a new one; used to record failures after a rollback.
23	CAP theorem?	In a network partition choose consistency or availability.
24	Isolation levels?	Read committed, repeatable read, serializable, increasing protection and cost.
25	What is MVCC?	Multiple row versions so readers do not block writers.

#	Question	Ten-second answer
26	Why index with partial predicate?	Smaller, faster and can express conditional uniqueness.
27	N+1 problem?	One query for parents plus one per child; fix with fetch join or entity graph.
28	Partitions vs consumers?	Parallelism is capped by partitions; extra consumers sit idle.
29	Backpressure?	Slow consumers limit intake: bounded queues, pull-based consumption.
30	Biggest weakness?	Complexity for the scale, and integration guarantees not yet proven by tests.
31	What would you add next?	Testcontainers tests, CDC, schema registry, jitter, circuit breakers.

J2. Whiteboard checklist (what to draw and say)

- **Draw**: API → Postgres (upload + outbox, one transaction) → relay → Kafka topics → four consumers → S3, with status topic → SSE and a failure branch to retry tiers, DLQ and incident engine.
- **Say the guarantees**: no lost event (outbox), no duplicate effect (idempotency), per-upload order (key), no blocked pipeline (retry topics), no double incident (partial unique index), no double cron (job_fires).
- **Say the caveats**: at-least-once transport, ordering only per key, poll-based relay, single broker locally, integration tests next.
- **Finish with scale**: partitions bound parallelism; outbox and DB are the first bottleneck; CDC, partitioned tables and lag-based autoscaling are the fixes.

J3. One-page cheat sheet

Area	Must-remember lines
Pitch	Event-driven pipeline: outbox → Kafka → idempotent stage consumers → versioned artifacts; failures classified, retried, dead-lettered and turned into deduplicated incidents.
Guarantees	No loss (outbox). No duplicate effect (processed_events in same tx). Order per upload (key). Stale skipped (version). One live incident (partial unique index). One cron fire (job_fires).
Kafka	Topic, partition, offset, consumer group, key. Ack after commit. Parallelism ≤ partitions. Retry 5s/1m/10m then DLQ. Rebalance means redelivery.
DB	ACID; MVCC; read committed default; unique constraint beats check-then-insert; SKIP LOCKED for queues; B-tree O(log n); partial indexes.
Spring	DI via constructor; proxies power @Transactional; self-invocation bypass; REQUIRES_NEW for failure recording; singletons stateless.
Security	BCrypt passwords; hashed expiring tokens; throttle; hashed reset codes with attempts; decoys; allow-listed tasks; department scoping.
Concurrency	Shared mutable state is the enemy; database arbitrates cross-process races; bounded queues give backpressure; consistent lock order avoids deadlock.
Scale	Estimate → bottleneck → fix → trade-off. First to break: DB writes and outbox; then partitions; then SSE.
Honesty	Domain tests pass (45); Spring/Kafka/JPA wiring needs mvn verify and compose run locally; integration tests are next.

J4. Before the interview: your 7-day plan

Day	Do this
1	Run mvn verify and docker compose up locally; fix what breaks; upload a file and watch it cross all stages.
2	Add one Testcontainers test: same event twice, assert a single effect. Now you have proof.

Day	Do this
3	Kill the worker mid-processing and show redelivery; screenshot Prometheus metrics.
4	Practise the 30-second and 2-minute pitches aloud; record yourself.
5	Whiteboard the architecture and the failure table without notes.
6	Drill Part F (DSA, DB, OS, networking) and the rapid-fire bank with a friend.
7	Do a mock interview where the friend interrupts with "what if it crashes here?"

Final advice

The single most impressive thing you can do is demonstrate a failure live: upload a bad file, show the retry tiers and the deduplicated incident, approve the destructive fixes, replay from the DLQ. A five-minute live demo of failure handling beats any slide.